

Data Analysis with Python

Cheat Sheet: Model Development

Process	Description	Code Example
Linear Regression	Create a Linear Regression model object	<pre>1. 1 2. 2 1. from sklearn.linear_model import LinearRegression 2. lr = LinearRegression()</pre>
Train Linear Regression model	Train the Linear Regression model on decided data, separating Input and Output attributes. When there is single attribute in input, then it is simple linear regression. When there are multiple attributes, it is multiple linear regression.	<pre>1. 1 2. 2 3. 3 1. X = df[['attribute_1', 'attribute_2', ...]] 2. Y = df['target_attribute'] 3. lr.fit(X,Y)</pre>
Generate output predictions	Predict the output for a set of Input attribute values.	<pre>1. 1 1. Y_hat = lr.predict(X)</pre>
Identify the coefficient and intercept	Identify the slope coefficient and intercept values of the linear regression model defined by $\hat{y} = mx + c$ Where m is the slope coefficient and c is the intercept.	<pre>1. 1 2. 2 1. coeff = lr.coef 2. intercept = lr.intercept_</pre>
Residual Plot	This function will regress y on x (possibly as a robust or polynomial regression) and then draw a scatterplot of the residuals.	<pre>1. 1 2. 2 3. 3 1. import seaborn as sns 2. sns.residplot(x=df[['attribute_1']], 3. y=df[['attribute_2']])</pre>
Distribution Plot	This function can be used to plot the distribution of data w.r.t. a given attribute.	<pre>1. 1 2. 2 3. 3 1. import seaborn as sns 2. sns.distplot(df['attribute_name'], hist=False) 3. # can include other parameters like color, label and so on.</pre>
Polynomial Regression	Available under the numpy package, for single variable feature creation and model fitting.	<pre>1. 1 2. 2 3. 3 4. 4 5. 5 6. 6 1. f = np.polyfit(x, y, n) 2. #creates the polynomial features of order n 3. p = np.poly1d(f) 4. #p becomes the polynomial model used to generate the predicted output 5. Y_hat = p(x) 6. # Y_hat is the predicted output</pre>
Multi-variate Polynomial Regression	Generate a new feature matrix consisting of all polynomial combinations of the features with the degree less than or equal to the specified degree.	<pre>1. 1 2. 2 3. 3 4. 4 1. from sklearn.preprocessing import PolynomialFeatures 2. Z = df[['attribute_1','attribute_2',...]] 3. pr=PolynomialFeatures(degree=n) 4. Z_pr=pr.fit_transform(Z)</pre>
Pipeline	Data Pipelines simplify the steps of processing the data. We create the pipeline by creating a list of tuples including the name of the model or estimator and its corresponding constructor.	<pre>1. 1 2. 2 3. 3 4. 4 5. 5 6. 6 7. 7 8. 8 9. 9 1. from sklearn.pipeline import Pipeline 2. from sklearn.preprocessing import StandardScaler 3. Input=[('scale',StandardScaler()), ('polynomial',</pre>

```
4. PolynomialFeatures(include_bias=False)),
5. ('model',LinearRegression()))]
6. pipe=Pipeline(Input)
7. Z = Z.astype(float)
8. pipe.fit(Z,y)
9. ypipe=pipe.predict(Z)
```

Copied!

a.

- 1. 1
- 2. 2
- 3. 3
- 4. 4

```
1. X = df[['attribute_1', 'attribute_2', ...]]
2. Y = df['target_attribute']
3. lr.fit(X,Y)
4. R2_score = lr.score(X,Y)
```

Copied!

b.

- 1. 1
- 2. 2
- 3. 3
- 4. 4

```
1. from sklearn.metrics import r2_score
2. f = np.polyfit(x, y, n)
3. p = np.poly1d(f)
4. R2_score = r2_score(y, p(x))
```

Copied!

- 1. 1
- 2. 2

```
1. from sklearn.metrics import mean_squared_error
2. mse = mean_squared_error(Y, Yhat)
```

Copied!

R^2 value

R^2, also known as the coefficient of determination, is a measure to indicate how close the data is to the fitted regression line. The value of the R-squared is the percentage of variation of the response variable (y) that is explained by a linear model.
a. For Linear Regression (single or multi attribute)
b. For Polynomial regression (single or multi attribute)

MSE value

The Mean Squared Error measures the average of the squares of errors, that is, the difference between actual value and the estimated value.

